OMICRON

Taster Day

of *Balint*

C1 - Internal

Hello Balint,

On the following pages you will find the tasks prepared for your Taster Day. These tasks reflect typical activities you would encounter in the role of **Software Quality Assurance Engineer with a focus on test automation**.

The example application used in this exercise is intentionally simplified. This ensures that you can run it locally on your own computer and complete the tasks using **publicly available tools and frameworks** such as **Visual Studio Code** and the **.NET SDK**, without the need for any proprietary software or special environments.

We are very interested in seeing how you approach the tasks, how you structure your work, and how you reason about quality and automation. There is no single “perfect” solution — we value your thought process and decisions just as much as the final result.

If you have any questions or need additional information at any point, please do not hesitate to contact us.

We wish you an enjoyable Taster Day and look forward to discussing your results with you.

Best regards

Mathias and the SWQAT Team

C1 - Internal

Topics/Tasks

1. User Story: Read the user story carefully.
    a. Define some test cases to properly test the requirements of the User Story
    b. What should be tested manually and how?
    c. What should be tested automatically? What level of automated test would you suggest (UI-Test, API-Test, Unit-Test)
2. Coding Example
    a. Set-Up Visual Studio Code and run the prepared example application (follow the steps on setup.txt)
    b. Review the code example, which implements the previously presented User Story
    c. Implement some of your previously defined automated test cases

---

C1 - Internal

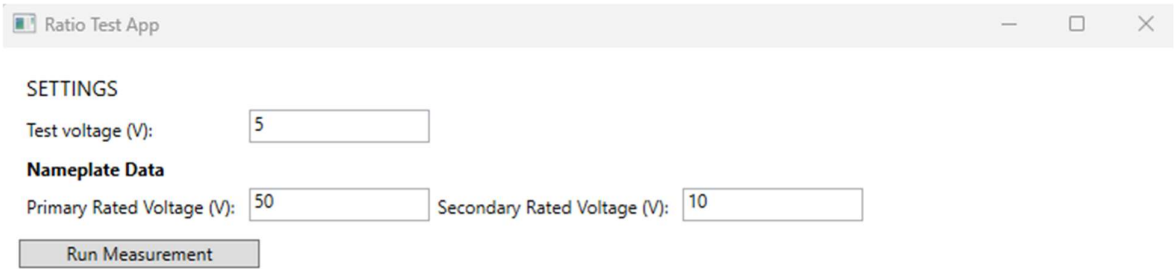
User Story: Transformer Ratio Test

Description

As a test engineer, I want to perform a voltage ratio test on a transformer by entering a test voltage and nameplate data, so that I can verify the transformer's turns ratio matches the expected value from its nameplate rating.

Requirements

Settings



Input — Test Voltage

The user can enter a test voltage (in volts). The value must be a positive number; an error message is shown if it is not.

Input — Nameplate Data

The user can enter the transformer's rated primary voltage and rated secondary voltage (both in volts, both must be positive numbers). These define the expected voltage ratio.

Expected Voltage Ratio = Primary Rated Voltage / Secondary Rated Voltage

Run Measurement

When the user clicks "Run Measurement", the application executes a ratio test using the supplied test voltage and nameplate values.

C1 - Internal

Results

After a successful measurement, the application shows measured inputs:

- Measured primary voltage (V)
- Measured primary current (A)
- Measured secondary voltage (V)

(NOTE: these measurements will be simulated in the example application)

And the calculated results

- Calculated voltage ratio (primary ÷ secondary)
- Ratio deviation (%) — the difference between the measured ratio and the expected nameplate ratio, expressed as a percentage

Validation & Error Feedback

If any input field contains an invalid or non-positive value, a descriptive error message is shown in red and no measurement is run.

<u>Example error, if test voltage is wrong:</u>

Test voltage must be a positive number. Maximum allowed is 42 V.

Result Cleared on New Run

All previous results and error messages are cleared each time the user initiates a new measurement.

C1 - Internal

Coding Example

1) Set-Up Visual Studio Code and run the prepared example application (follow the steps on setup.txt)
2) Review the code example, which implements the previously presented User Story
3) Implement some of your previously defined automated test cases (UI-Test, API-Test or Unit-Test).

Preferred tools, Libraries (when needed)

- XUnit
- FlaUI

If the code is not testable for your concept, feel free to refactor the code (please document your changes and explain, what and why it was optimized)

C1 - Internal